

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студент гр. 4343

И.И. Иванов

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: **И.И. Иванов**

Группа: 4343

Тема практики: Генетические алгоритмы

Задание на практику:

Дан взвешенный неориентированный граф (ребра имеют вес больше 0).
Необходимо найти минимальное остовное дерево для данного графа,
используя генетические алгоритмы.

Сроки прохождения практики: 06.07.2026 – 17.07.2026

Дата сдачи отчета: **00.00.0000**

Дата защиты отчета: **00.00.0000**

Студент

И.И. Иванов

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

В отчете представлены результаты выполнения учебной практики, посвященной исследованию и реализации генетического алгоритма для решения задачи поиска минимального остовного дерева взвешенного неориентированного графа. Он содержит описание предметной области, постановку задачи, результаты разработки, сведения об использованных технологиях и выводы по итогам выполненной работы.

SUMMARY

This report presents the results of an academic project focused on researching and implementing a genetic algorithm to solve the problem of finding the minimum spanning tree of a weighted undirected graph. It includes a description of the problem domain, a problem statement, development results, details regarding the technologies used, and conclusions based on the work performed.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	8
1.1 Предметная область	8
1.2 Задачи практики	8
1.2.1 Изучение предметной области	8
1.2.2 Анализ требований к программному средству.....	9
1.2.3 Проектирование структуры программного решения	9
1.2.4 Проектирование генетического алгоритма.....	9
1.2.5 Реализация алгоритма	9
1.2.6 Разработка графического пользовательского интерфейса.....	9
1.2.7 Реализация способов задания исходных данных	10
1.2.8 Тестирование программного средства	10
1.2.9 Анализ результатов	10
1.3 Проектирование программного решения.....	10
1.3.1 Формат хранения данных	10
1.3.2 Представление решения.....	11
1.3.3 Функция исправления решения.....	11
1.3.4 Функция приспособленности	12
1.3.5 Выбор родителей.....	12
1.3.6 Оператор скрещивания	13
1.3.7 Оператор мутации	13
1.3.8 Критерий завершения алгоритма	14
1.4 Проектирование графического интерфейса.....	14
2 РЕЗУЛЬТАТЫ РАБОТЫ	15
2.1. Формулировка задачи 1.....	15
2.2. Формулировка задачи 2.....	15
2.3. Формулировка задачи 3.....	15
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	16
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	17
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	19

ПРИЛОЖЕНИЕ А	20
--------------------	----

ВВЕДЕНИЕ

Предметной областью учебной практики являются методы решения задач комбинаторной оптимизации с использованием генетических алгоритмов, которые моделируют процессы естественного отбора и наследования для поиска оптимальных или близких к оптимальным решений. В рамках практики рассматривается задача поиска минимального остовного дерева взвешенного неориентированного графа, позволяющая продемонстрировать применение таких алгоритмов при решении сложных оптимизационных задач.

Целью учебной практики является разработка программного средства, реализующего поиск минимального остовного дерева с использованием генетического алгоритма, а также исследование особенностей его работы и визуализация процесса поиска решения.

Для достижения поставленной цели необходимо решить следующие задачи:

- Изучить теоретические основы генетических алгоритмов и особенности их применения при решении задач комбинаторной оптимизации;
- Изучить методы поиска минимального остовного дерева и определить особенности применения генетического алгоритма для решения данной задачи;
- Разработать способ представления графа и кодирования решения в виде хромосомы;
- Определить функцию приспособленности и выбрать генетические операторы отбора, скрещивания и мутации;
- Разработать архитектуру программного средства и определить взаимодействие основных программных модулей;
- Реализовать алгоритм поиска минимального остовного дерева с использованием генетического алгоритма;

- Разработать графический пользовательский интерфейс, обеспечивающий ввод исходных данных, настройку параметров алгоритма, управление процессом вычислений и визуализацию промежуточных и конечных результатов;
- Реализовать возможность задания исходных данных посредством ручного ввода, чтения из файла и случайной генерации графа;
- Выполнить тестирование разработанного программного средства на различных наборах данных и провести анализ полученных результатов.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Одной из важных задач теории графов является поиск минимального остовного дерева взвешенного графа. Остовным деревом называется подграф, содержащий все вершины исходного графа, не имеющий циклов и состоящий из минимально необходимого количества ребер. Минимальным остовным деревом является такое остовное дерево, суммарный вес ребер которого минимален среди всех возможных остовных деревьев данного графа.

Традиционно данная задача эффективно решается классическими алгоритмами, такими как алгоритмы Краскала и Прима. Однако использование генетических алгоритмов позволяет рассмотреть задачу с точки зрения методов оптимизации, основанных на моделировании процессов естественного отбора, наследования и мутации.

В рамках данной практики рассматривается реализация генетического алгоритма поиска минимального остовного дерева взвешенного неориентированного графа. Алгоритм должен обеспечивать формирование начальной популяции решений, вычисление функции приспособленности, выполнение операций отбора, скрещивания и мутации, а также последовательное улучшение найденных решений до достижения критерия остановки. Дополнительно предусматривается визуализация процесса работы алгоритма и анализ изменения качества решений по поколениям.

1.2 Задачи практики

1.2.1 Изучение предметной области

Изучить особенности задачи поиска минимального остовного дерева взвешенного неориентированного графа, рассмотреть существующие алгоритмы ее решения, а также исследовать принципы работы генетических алгоритмов и возможность их применения к задачам комбинаторной оптимизации.

1.2.2 Анализ требований к программному средству

Проанализировать требования, предъявляемые к разрабатываемому программному обеспечению, определить способы задания исходных данных, параметры настройки генетического алгоритма, требования к визуализации процесса поиска решения и отображению результатов работы.

1.2.3 Проектирование структуры программного решения

Разработать архитектуру программного средства, определить основные классы, структуры данных и взаимодействие отдельных модулей, отвечающих за представление графа, выполнение генетического алгоритма и взаимодействие с графическим интерфейсом.

1.2.4 Проектирование генетического алгоритма

Разработать структуру генетического алгоритма поиска минимального остова дерева, определить способ кодирования решений, выбрать функцию приспособленности, методы отбора, скрещивания и мутации, а также установить критерий завершения работы алгоритма.

1.2.5 Реализация алгоритма

Разработать программный модуль, реализующий генерацию начальной популяции, вычисление функции приспособленности, выполнение генетических операторов, формирование новых поколений и поиск наилучшего решения.

1.2.6 Разработка графического пользовательского интерфейса

Разработать графический пользовательский интерфейс, обеспечивающий удобное взаимодействие пользователя с программой. Интерфейс должен предоставлять возможность создания и редактирования графа, загрузки данных из файла, случайной генерации входных данных, настройки параметров генетического алгоритма, запуска и пошагового выполнения вычислений, отображения текущего состояния алгоритма, найденного остова дерева, а также графика изменения функции качества по поколениям.

1.2.7 Реализация способов задания исходных данных

Обеспечить возможность задания исходного графа несколькими способами: ручным вводом через графический интерфейс, чтением из файла и случайной генерацией графа.

1.2.8 Тестирование программного средства

Провести тестирование разработанного программного обеспечения на различных входных данных, проверить корректность работы генетического алгоритма и графического интерфейса.

1.2.9 Анализ результатов

Проанализировать результаты работы разработанного программного средства, оценить эффективность выбранных методов и сделать выводы о возможности применения генетического алгоритма для решения задачи поиска минимального остовного дерева.

1.3 Проектирование программного решения

1.3.1 Формат хранения данных

Для хранения исходного графа выбрана структура данных, основанная на списке ребер. Такое представление является универсальным, компактным и удобно используется как при чтении графа из файла, так и при случайной генерации или вводе пользователем через графический интерфейс.

Каждое ребро графа содержит номера двух соединяемых вершин и вес ребра. В программной реализации ребро описывается структурой:

```
struct Edge
{
    int from;
    int to;
    double weight;
};
```

Полный граф представляет собой количество вершин и список всех ребер:

```
struct Graph
{
    int vertexCount;
    std::vector<Edge> edges;
};
```

Данный способ хранения позволяет быстро получать информацию о каждом ребре и использовать его индекс при построении хромосом генетического алгоритма.

Исходные данные также могут храниться в текстовом файле. В первой строке располагается количество вершин и количество ребер, после чего перечисляются все ребра графа в формате: $u \ v \ weight$. Такой формат является простым для чтения, легко редактируется вручную и удобен для тестирования программы.

1.3.2 Представление решения

В данной работе каждая особь представляет собой список индексов ребер исходного графа. Таким образом, каждая хромосома определяет набор ребер, который рассматривается как кандидат на минимальное остовное дерево.

Например, если граф содержит девять ребер с индексами $\{0,1,2,3,4,5,6,7,8\}$, то особь $\{0,3,5,7,8\}$ означает, что предполагаемое дерево состоит именно из этих пяти ребер.

Такой способ кодирования обладает несколькими преимуществами: простотой реализации, небольшим объемом памяти, удобством выполнения генетических операторов, независимостью от конкретного расположения вершин графа.

Недостатком данного представления является возможность появления недопустимых решений. После операций скрещивания или мутации выбранный набор ребер может содержать циклы либо не соединять все вершины графа. Для устранения подобных ситуаций используется специальная процедура исправления решения.

1.3.3 Функция исправления решения

После выполнения операций скрещивания и мутации каждая новая особь проходит процедуру исправления. Назначением данной процедуры является преобразование произвольного набора ребер в корректное остовное дерево.

Работа функции выполняется в два этапа:

- На первом этапе производится поиск циклов. Если цикл обнаружен, из него удаляется ребро с наибольшим весом, поскольку именно оно вносит наибольший вклад в стоимость дерева.
- На втором этапе проверяется связность полученного графа. Если граф распался на несколько компонент связности, между ними добавляется ребро минимального веса. Процесс продолжается до тех пор, пока все вершины не окажутся соединенными.

1.3.4 Функция приспособленности

Качество каждой особи оценивается функцией приспособленности. Поскольку целью задачи является минимизация суммарного веса остовного дерева, в качестве оценки используется обратная зависимость между стоимостью дерева и его приспособленностью.

Пусть W – суммарный вес ребер дерева; MAX_WEIGHT – некоторое достаточно большое постоянное значение. Тогда функция приспособленности определяется следующим образом: $fitness = MAX_WEIGHT - W$.

Таким образом, чем меньше общий вес найденного дерева, тем больше значение функции приспособленности, а следовательно, тем выше вероятность участия данной особи в формировании следующего поколения.

1.3.5 Выбор родителей

Для формирования родительских пар выбран турнирный отбор.

Суть метода заключается в том, что из текущей популяции случайным образом выбирается небольшая группа особей, после чего в качестве родителя выбирается особь с наибольшим значением функции приспособленности.

Выбор данного метода обусловлен несколькими причинами:

- Простотой программной реализации;
- Отсутствием необходимости вычислять вероятности выбора каждой особи;

- Устойчивостью к различным диапазонам значений функции приспособленности. По сравнению с рулеточным отбором турнирный метод менее чувствителен к масштабу функции качества и обеспечивает стабильную работу алгоритма;
- Хорошим балансом между исследованием новых решений и сохранением лучших найденных особей.

1.3.6 Оператор скрещивания

Для получения новых решений используется равномерное скрещивание. Для каждого гена независимо определяется, от какого родителя он будет унаследован. При формировании первого потомка для каждого гена с одинаковой вероятностью выбирается значение первого или второго родителя. Второй потомок получает противоположный выбор.

Основным преимуществом такого метода является возможность свободного комбинирования отдельных удачных ребер из различных решений. В отличие от одноточечного или двухточечного скрещивания отсутствует привязка к расположению генов внутри хромосомы, что особенно важно при решении задачи поиска минимального остовного дерева, где порядок расположения ребер не оказывает влияния на качество решения.

После завершения операции скрещивания выполняется процедура исправления решения.

1.3.7 Оператор мутации

Мутация предназначена для поддержания разнообразия популяции и предотвращения преждевременной сходимости алгоритма.

В разработанном алгоритме мутация выполняется следующим образом:

- Случайным образом удаляется одно ребро;
- Случайным образом выбирается новое ребро исходного графа и добавляется в хромосому;
- Выполняется процедура восстановления корректного дерева.

Такой подход позволяет исследовать новые области пространства поиска без существенного нарушения структуры найденных решений.

1.3.8 Критерий завершения алгоритма

Для завершения работы генетического алгоритма используется комбинированный критерий остановки.

Алгоритм прекращает работу при выполнении одного из двух условий:

- Достигнуто максимальное количество поколений;
- Лучшее найденное решение не улучшается в течение заданного количества поколений подряд.

Использование сразу двух критериев позволяет, с одной стороны, гарантировать завершение работы программы, а с другой – избежать выполнения лишних вычислений после фактической сходимости популяции.

1.4 Проектирование графического интерфейса

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Формулировка задачи 1

2.2. Формулировка задачи 2

2.3. Формулировка задачи 3

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

4 ОТЗЫВ РУКОВОДИТЕЛЯ

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.

ПРИЛОЖЕНИЕ А